

UNIVERSIDAD NACIONAL DE LA PLATA
FACULTAD DE INFORMÁTICA — INTERNET DE LAS COSAS 2026

Sistema de Geofencing para Ganado con Enlace LoRa de Bajo Consumo y Alertas

Autores: Cid De La Paz, Alejo — Pacchialat, Taciano

Resumen

El presente informe técnico describe el diseño, arquitectura e implementación de una plataforma de monitoreo de ubicación y delimitación perimetral virtual (geofencing) para ganado bovino en entornos rurales sin cobertura de telefonía celular. El sistema articula nodos móviles de borde (collares) basados en microcontroladores ESP32 con receptores GPS y transceptores LoRa SX1278 a 433 MHz operando bajo ciclos estrictos de temporización en *Deep Sleep*. Los paquetes de telemetría son capturados por una compuerta de enlace (*Gateway*) ESP32 y retransmitidos vía MQTT e InfluxDB hacia un servidor local Node-RED y contenedores Docker. La verificación espacial punto-en-polígono (*Ray-Casting*) y el control paramétrico se gestionan desde una interfaz de monitoreo y un bot bidireccional de Telegram con alertas proactivas.

1 Introducción y Objetivos

La gestión y localización de ganado bovino en extensiones rurales extensas presenta restricciones operativas críticas vinculadas a la ausencia de infraestructura eléctrica y de cobertura de telefonía celular comercial. Los sistemas tradicionales de delimitación física con alambrado perimetral o boyeros eléctricos incurren en elevados costos de mantenimiento e imposibilitan la modificación dinámica del área de pastoreo.

El presente proyecto implementa una plataforma de delimitación geográfica virtual por software (*geofencing*) integrada con telemetría de posicionamiento por satélite y un enlace de comunicación *LoRa*. El sistema permite monitorear las coordenadas de cada animal y generar alertas ante salidas perimetrales o fallas de conectividad, operando de forma autónoma sin depender de redes de terceros en el campo.

1.1 Objetivos del Sistema

Desarrollo de nodo de borde de bajo consumo: Diseñar un collar móvil autónomo basado en el microcontrolador ESP32 con receptor GPS simulado y transceptor LoRa SX1278 (433 MHz), operando bajo ciclos temporizados en modo *Deep Sleep* para maximizar la vida útil de la batería.

Enrutamiento local: Implementar un nodo de *Gateway* capaz de capturar paquetes de LoRa e inyectarlos a una red Wi-Fi local mediante el protocolo MQTT.

Procesamiento espacial: El cálculo algebraico del perímetro se realiza en el microcontrolador del collar, ejecutando el algoritmo de pertenencia poligonal, que luego se sincroniza con un servidor central local (Node-RED) con acceso a almacenamiento histórico y temporal de InfluxDB.

Control remoto: Proveer una interfaz web interactiva con representación en mapa para visualiza-

ción de telemetría y edición paramétrica de límites del Geofence, complementada con un bot bidireccional de Telegram para alertas e interrogación de estado.

2 Arquitectura General y Selección de Hardware

La topología del sistema adopta una estructura jerárquica de tres capas: capa de borde (collares móviles en el ganado), capa de agregación (Gateway LoRa/Wi-Fi), y capa de backend (servidor local con contenedores Docker).

2.1 Diagrama de Bloques del Sistema

La Figura 1 detalla la interconexión de componentes de hardware y los protocolos de comunicación utilizados en cada tramo del flujo de datos.

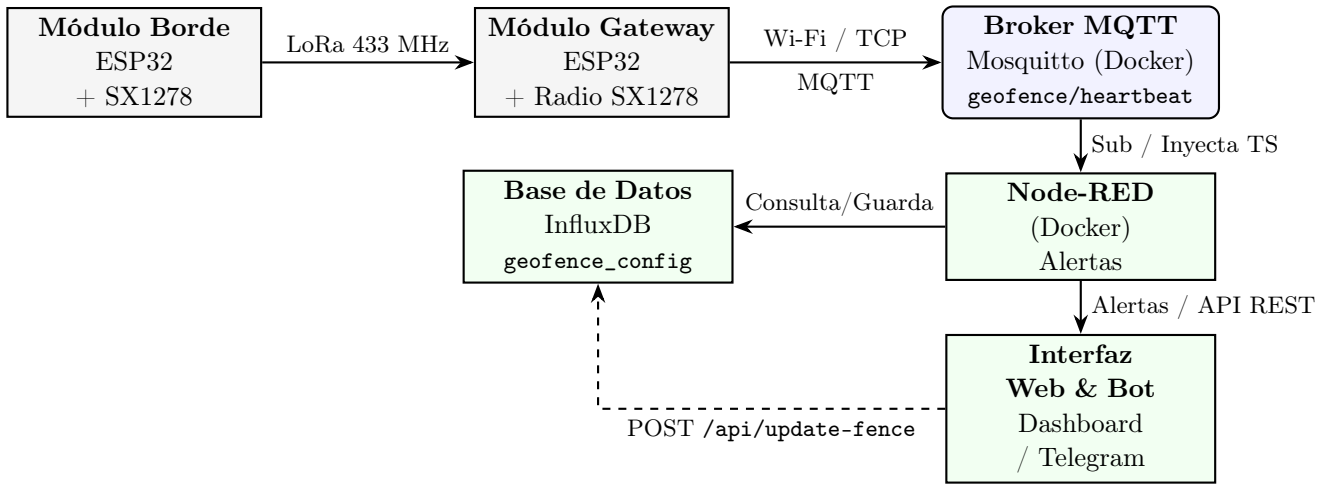


Figura 1: Topología general del sistema de geofencing ganadero y flujo de información entre capas.

2.2 Especificaciones Técnicas del Hardware

Para satisfacer las restricciones de consumo en el collar y las exigencias de conectividad, se seleccionaron dos variantes del microcontrolador ESP32 de Espressif Systems combinadas con transceptores de radiofrecuencia. La Tabla 1 sistematiza las especificaciones operativas y la justificación de diseño para cada componente.

Componente	Modelo / IC	Especificación Principal	Función en el Sistema
MCU Borde	ESP32-WROOM-32D	Dual-core Xtensa LX6, 240 MHz	Procesamiento en collar, temporización con reloj de tiempo real (RTC) en Deep Sleep.
MCU Gateway	ESP32-WROOM-32D	Dual-core Xtensa LX6, 240 MHz	Receptor base de interrupciones de radio e interfaz Wi-Fi hacia el servidor local.
Radio LoRa	SX1278	Frecuencia 433 MHz	Enlace RF de largo alcance (> 4 km en línea de vista) con modulaciones de espectro ensanchado.
Receptor GPS	Simulado	Mediante comandos UART al MCU	Adquisición de longitud y latitud.

Tabla 1: Especificaciones y asignación de hardware del sistema.

El enlace de radiofrecuencia opera en la banda ISM de 433 MHz con modulación LoRa. Esto garantiza un balance entre inmunidad al ruido electromagnético en zonas abiertas, velocidad de transmisión y tiempo en el aire, reduciendo el consumo energético en cada ráfaga de telemetría.

3 Desarrollo y Arquitectura del Firmware

El firmware del sistema de geofencing está estructurado en dos proyectos independientes desarrollados en C++ bajo el framework de Espressif (*ESP-IDF*): el módulo móvil de borde (*edge-module*) incrustado en el collar del ganado y el módulo base de enlace (*gateway-module*). La arquitectura de ambos nodos prioriza la estabilidad de ejecución continua y el manejo eficiente de recursos temporales y de memoria.

3.1 Módulo Edge

El collar opera bajo restricciones energéticas. Para maximizar su autonomía sin perder capacidad de respuesta o trazabilidad espacial, el firmware implementa una máquina de estados temporizada regida por el controlador del Reloj de Tiempo Real (RTC) del ESP32. En el prototipo se alimentó al MCU mediante USB hacia la computadora, pero se realizó con la intención de tener una gran autonomía si fuera alimentado con baterías.

Durante la mayor parte de su ciclo operativo, el microcontrolador permanece en estado de bajo consumo profundo (*Deep Sleep*), donde se desenergizan los núcleos de CPU, el controlador de radio LoRa y los periféricos estándar, reduciendo el consumo total a (μA). Para mantener la coherencia de telemetría entre ciclos de sueño sin acceder al almacenamiento no volátil Flash (lo que incrementaría el consumo y desgastaría el medio), las variables de control y contadores de secuencia se almacenan en la memoria estática lenta del RTC (*RTC_DATA_ATTR*). En este dominio de memoria persisten el contador incremental de despertares (*boot_count*), las últimas coordenadas validadas y el estado actual de alarma del cerco.

El temporizador del RTC activa el sistema periódicamente cada 30 segundos. La Figura 2 ilustra el flujo secuencial de tareas ejecutado en cada ciclo de *Wakeup*.



Figura 2: Diagrama de estados del módulo de borde.

Al salir de *Deep Sleep*, la secuencia operativa se desarrolla de la siguiente manera:

1. **Inicialización:** El procesador verifica la causa de activación del RTC y recupera las variables de estado desde el registro `RTC_DATA_ATTR`.
2. **Adquisición:** Se habilita la interfaz UART para capturar las coordenadas simuladas. Si el módulo obtiene una solución de posición válida, extrae la longitud y latitud. Si se agota el tiempo máximo de espera sin fijación, el sistema utiliza la última coordenada conocida y marca un indicador de telemetría degradada.
3. **Transmisión:** El procesador empaqueta los datos posicionales, el indicador de batería, el contador de secuencias y el estado local en una estructura y ordena su emisión a través del bus SPI hacia el transceptor LoRa SX1278.
4. **Ventana de recepción:** Tras finalizar la transmisión del paquete, el transceptor conmuta inmediatamente a modo recepción durante una ventana temporal estricta de 2000 milisegundos (2,0 segundos). Esta ventana sincronizada permite que el Gateway transmita actualizaciones de parámetros del cerco sin obligar al collar a mantener el receptor encendido de forma permanente. Si el transceptor detecta un paquete entrante válida durante la ventana, la lógica interpreta el comando. Si la orden instruye una alarma perimetral, se envía en el próximo ciclo las nuevas coordenadas y el estado de FUERA.
5. **Retorno:** Concluida la ventana de recepción y ejecutada la actuación local, el microcontrolador reconfigura el temporizador de alarma del RTC y desenergiza todos los buses, retornando al modo *Deep Sleep*.

3.2 Módulo Gateway

El módulo Gateway opera de forma estacionaria y alimentado de forma continua. Su responsabilidad consiste en escuchar permanentemente el canal LoRa, recibir las ráfagas de telemetría entrantes, formatear las tramas en objetos JSON estructurados y publicarlos a través de Wi-Fi hacia el broker MQTT.

El transceptor SX1278 notifica la llegada de un paquete completo activando la línea digital de interrupción externa DI00. Para garantizar ejecución la ISR se restringe estrictamente a la memoria interna RAM del procesador mediante el atributo de compilador `IRAM_ATTR`, prohibiendo accesos indirectos a la memoria Flash externa que podrían generar demoras por fallas de caché. La ISR se limita a limpiar el flag de interrupción del hardware del radio y emitir una notificación directa hacia una tarea dedicada de FreeRTOS de alta prioridad.

El planificador del FreeRTOS desbloquea de inmediato la tarea manejadora de recepción LoRa al recibir la señal de la ISR. Esta tarea dedicada realiza la lectura de los registros del transceptor vía SPI, extrae la carga útil del paquete y deposita la estructura resultante en una cola de mensajes segura para concurrencia.

De forma concurrente, una segunda tarea de comunicación de red extrae los paquetes del búfer, los

serializa y los transmite mediante MQTT hacia el broker.

4 Backend

El procesamiento de servidor y el almacenamiento de datos operan sobre un ecosistema de micro-servicios contenidos mediante Docker en un servidor local.

4.1 Stack Docker

El servidor ejecuta cuatro contenedores interconectados en una red virtual privada aislada:

Broker Mosquitto: Actúa como concentrador MQTT local recibiendo las publicaciones del Gateway en el tópic `geofence/heartbeat` y notificando cambios perimetrales en `geofence/config_update`.

Base de Datos InfluxDB: almacena las posiciones históricas en la medición `cattle_position` y la configuración del perímetro en `geofence_config`.

Flujos Node-RED: enrutamiento de paquetes, decodificación JSON, validación espacial de la geofence y exposición de endpoints.

4.2 APIs

La configuración del cerco virtual se almacena de manera persistente dentro de la medición `geofence_config` de InfluxDB, conservando el cerco activo ante reinicios del servidor o de los servicios de Docker. La Tabla 2 detalla las especificaciones técnicas de los puntos de entrada expuestos por Node-RED y enrutados a través de Nginx.

Método	Ruta (/api/)	Carga Útil (Payload)	Comportamiento Técnico y Persistencia
GET	<code>fence/current</code>	<i>Ninguno</i>	Consulta el último registro en <code>geofence_config</code> de InfluxDB. Retorna objeto con <code>vertices</code> JSON y coordenadas del bounding box. Si no hay datos, emite polígono por defecto.
POST	<code>update-fence</code>	JSON: { <code>vertices</code> : [{ <code>lat</code> , <code>lng</code> }]}	Valida arreglo de coordenadas, calcula límites geográficos mínimos y máximos (<code>min/max lat/lon</code>), inserta nuevo punto en <code>geofence_config</code> y publica notificación en tópic MQTT <code>geofence/config_update</code> .
POST	<code>fence</code>	JSON: { <code>vertices</code> : [{ <code>lat</code> , <code>lng</code> }]}	Endpoint de compatibilidad directa con las vistas de edición web, enrutado al manejador transaccional de <code>update-fence</code> .

Tabla 2: Especificación APIs.

5 Interfaz de Usuario

La interacción del operador con el sistema se realiza a través de dos canales: un dashboard y bot de Telegram. Ambas interfaces operan sobre flujos de datos asincrónicos.

5.1 Dashboard

El tablero de control web funciona como una aplicación de página única, estructurada sobre primitivas de HTML5, JavaScript moderno (ES6+) y CSS nativo sin dependencias pesadas.

La representación gráfica del terreno y de las unidades ganaderas se ejecuta mediante la biblioteca de *Leaflet.js*. Al iniciar la sesión, el cliente realiza una petición HTTP inicial hacia el endpoint GET `/api/fence/current` para obtener el cerco activo de la base de datos y trazar el polígono

delimitador sobre la capa del mapa.

De manera paralela, la aplicación establece un canal de comunicación dúplex persistente mediante *WebSocket* contra el motor Node-RED. Cada vez que el Gateway ingesta un nuevo paquete de telemetría por radio LoRa, el servidor retransmite el evento `update_location` a los clientes web conectados. La lógica del frontend actualiza de inmediato las coordenadas del marcador correspondiente al animal, la información del enlace, el nivel de voltaje de la batería y la última marca de tiempo de sincronización.

Para permitir un ajuste de los límites del perímetro virtual sobre el terreno, la interfaz incorpora un panel lateral interactivo de vértices que habilita la modificación de la geocerca mediante eventos de teclado.

Al finalizar el ajuste, el operador acciona el comando de guardado, lo que desencadena la emisión de una petición `POST /api/update-fence`. El servidor recibe la nueva geometría, recalcula la caja (*Bounding Box*) y actualiza en milisegundos las reglas de validación en todos los nodos de monitoreo activos.

La supervisión remota requiere un mecanismo de notificación push que no dependa de mantener un navegador web abierto continuamente. Para ello, el backend integra un bot de Telegram gestionado por la biblioteca `node-telegram-bot-api` dentro del entorno de Node-RED.

Cuando el algoritmo determina que la posición censada de un animal sale del cerco (`inside === false`), el módulo de notificaciones consulta en InfluxDB las últimas coordenadas reportadas del identificador del collar y estructura un mensaje de advertencia hacia los operadores.

El bot de Telegram opera como terminal de comandos remota, permitiendo al administrador del sistema intervenir de manera directa sobre los actuadores físicos del collar sin necesidad de acceder a la consola de servidor o al tablero web.

6 Conclusiones y Trabajo Futuro

El desarrollo y validación de este sistema demostró la viabilidad técnica de implementar monitoreo geoespacial sin cobertura celular. La arquitectura dividida en tres capas permitió aislar las restricciones energéticas del collar (Módulo Edge) mediante un enlace de radiofrecuencia LoRa en 433 MHz hacia un Módulo Gateway, el cual procesa y deriva la telemetría a una infraestructura centralizada basada en Docker, Node-RED e InfluxDB.

Desde la perspectiva algorítmica y de firmware, se resolvieron problemas críticos de concurrencia y consumo. El uso de la memoria estática del reloj de tiempo real `RTC_DATA_ATTR` conservó el contexto de operación durante los ciclos de *Deep Sleep*. Asimismo, la inyección de marcas de tiempo en el servidor y la optimización por *Bounding Box* en el cálculo de del posicionamiento en la cerca aseguraron una detección de fugas certera.